Lecture 1-2

27 July 2026 16:05

CS6013: ADSA

1. Asymptotic Notation
2. Basics of BST - insertion/deletion in BST
3. AVL tree, Red Black tree
4. Hashing
5. Graph Algorithms
6. Algorithm Design Paradigm — Divide & Conquer, Greedy, DP
7. Basics of NP completeness, Reduction
8. Time permits FFT etc

Evaluation Policy

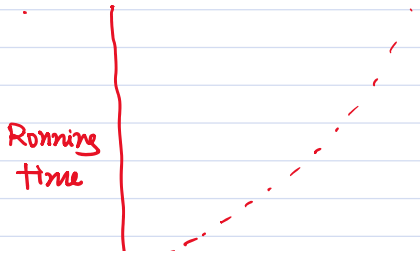
1. Midsem 30%.
2. Endsem 30%
3. Assignments (theory + coding) 10%
4. Lab exam 25%. (Oct - Nov)
5. Attendance 5%.

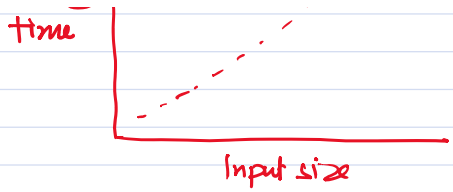


Ex: Set of real nos of size 100 — Output the given nos in sorted order

Efficient algo.

- lesser time and memory





Limitation of Empirical Running time

① Depends of S/W and H/W platform that is used

② Experiment can be done on limited set of Inputs. this may not be indicative of running time on other inputs that were not considered

③ Implement the algo

Remedy - Develop a general methodology of analysing running time of algo

- Use high level description of algo (instead of implementation)
- take care of all possible inputs
- Allow to capture running time indep of S/W & H/W platform used

We will capture running time as a fn of input size.

Count the no. of primitive operation

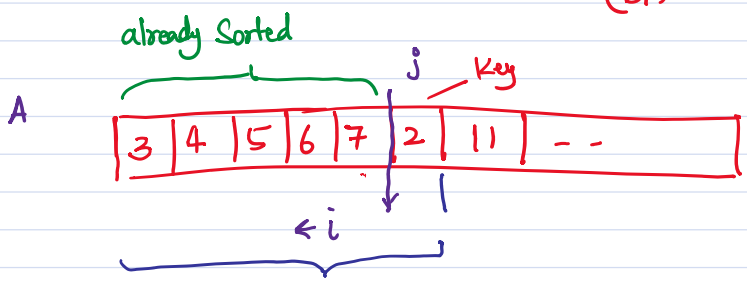
- Data movement (assignment)
- Arithmetic & logic operation (addition, comparision)
- Control (Branch ..)

Example (Sorting)

Input
Sequence of No.
$a_1 \; -- \; a_n$

Output
Seq of no. $(b_1 \; -- \; b_n)$ s.t
$b_1 \leq b_2 \; -- \; \leq b_n$

and $(b_1, \; -- \; b_n)$ is a permutation of $(a_1 \; -- \; a_n)$



Insertion Sort

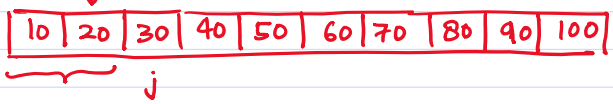
Cost # of iteration

For $i \leftarrow 2$ to n do	c_1	$n-1$
$key \leftarrow A[j]$	c_2	$n-1$
// Insert key into sorted seq $A[1 \ldots j-1]$		
$i \leftarrow j-1$	c_3	$n-1$
while $i>0$ and $A[i] > key$	c_4	$\sum_{j=2}^{n} t_j$
do $A[i+1] \leftarrow A[i]$	c_5	$\sum_{j=2}^{n} t_j$
$i--$	c_6	"
$A[i+1] \leftarrow key$	c_7	$(n-1)$

t_j := time required to put jth element of array in correct position of sorted order

$$\text{Total time} = (c_1+c_2+c_3+c_7)(n-1) + (c_4+c_5+c_6)\sum_{j=2}^{n} t_j$$

Input



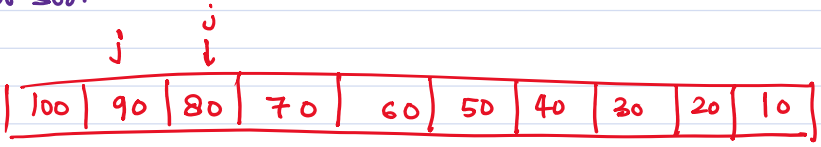
When Input is already sorted $t_j = 1$

$$\text{Total time} = (c_1+c_2+c_3+c_7)(n-1) + (c_4+c_5+c_6)\sum_{j=2}^{n} 1$$

$$= (n-1)\,C \qquad \sum_{i=1}^{7} c_i = C$$

Best Case for Insertion Sort

Consider Input



Input is sorted in reverse order $t_j = (j-1)$

$$\text{Total time} = (c_1+c_2+c_3+c_7)(n-1) + (c_4+c_5+c_6)\sum_{j=2}^{n} t_j$$

$$= (c_1+c_2+c_3+c_7)(n-1) + (c_4+c_5+c_6)\sum_{j=2}^{n} (j-1)$$

Worst Case

$$= \underbrace{(c_1+c_2+c_3+c_7)}_{c'}(n-1) + \underbrace{(c_4+c_5+c_6)}_{c''}\frac{n(n-1)}{2}$$

$$= c'(n-1) + c''\frac{n(n-1)}{2}$$

$$= c'(n-1) + c'' \frac{n(n-1)}{2}$$

Average Case $t_j = j/2$

$$\text{Total time} = (c_1+c_2+c_3+c_7)(n-1) + (c_4+c_5+c_6)\sum_{j=2}^{n} t_j$$

$$= (c_1+c_2+c_3+c_7)(n-1) + (c_4+c_5+c_6)\sum_{j=2}^{n} j/2 = f(n^2)$$

Worst case is generally used

$$f(n) = (n-1)c + (n^3-n^2+5)c_1 + e^{\sqrt{n}} \log n$$

Constt, log, linear, Qu . , poly in n , exp.

Worst case running time is generally used

① Computing avg case running time might be difficult
- need to have prob distribution on all possible input instance } difficult
- compute expected running time

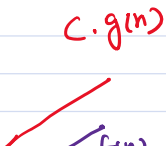
② Avg case could be as bad as worst case

Asymptotic Analysis

Goal: To simplify analysis of running time by getting rid of details that depends on s/w or h/w platform used

Focus on case as $n \to \infty$

Asymptotic Upper Bound (Big Oh notation)

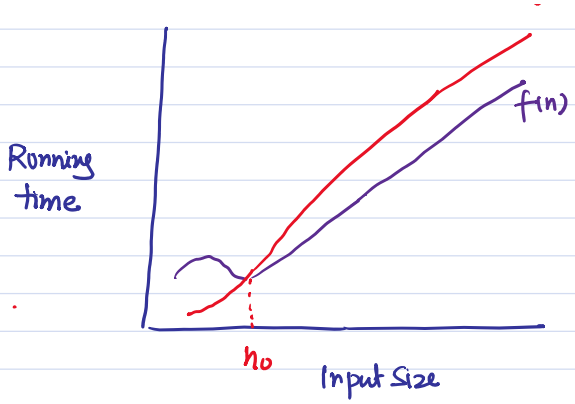


Asymptotic Upper Bound (Big Oh notation)

$f(n)$ fn capture running time of algo.
$g(n)$ clean fn

$f(n) = O(g(n))$ if $\exists$ a constt c and n_0 s.t

$\forall \; n \geq n_0$ we have $f(n) \leq c \cdot g(n)$



Both $f(n)$ and $g(n)$ are non-negative fn over non-neg input.

"f is no larger than g"

Ex $f(n) = 10n^2 + 5n + 100$ $\quad 5n \leq 5n^2 \quad n \geq 1$

$\leq 10n^2 + 5n^2 + 100 \quad n \geq 1$

$\leq 10n^2 + 5n^2 + 100n^2$

$= 115n^2$

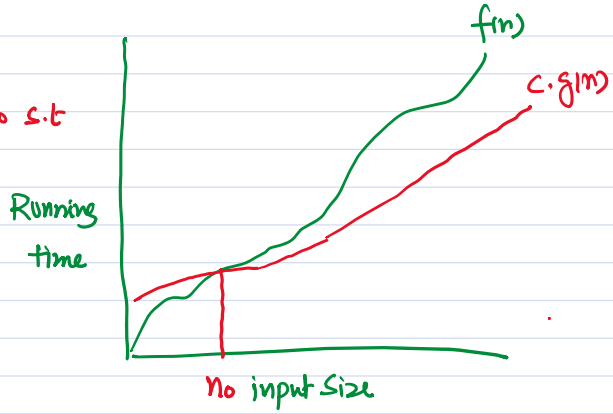
$f(n) \leq 115\, n^2 \quad n \geq n_0 = 1 \quad c = 115$

$\Rightarrow f(n) = O(n^2) \quad f(n) \in O(n^2)$

Asymptotic lower Bound (Big Omega notation)

$f(n) = \Omega(g(n))$ if $\exists$ a constt c and n_0 s.t

$c \cdot g(n) \leq f(n) \quad \forall \; n \geq n_0$



Ex. $f(n) = 10n^2 + 5n + 100$

$\geq 10n^2$

$c = 10, \; n_0 = 1$

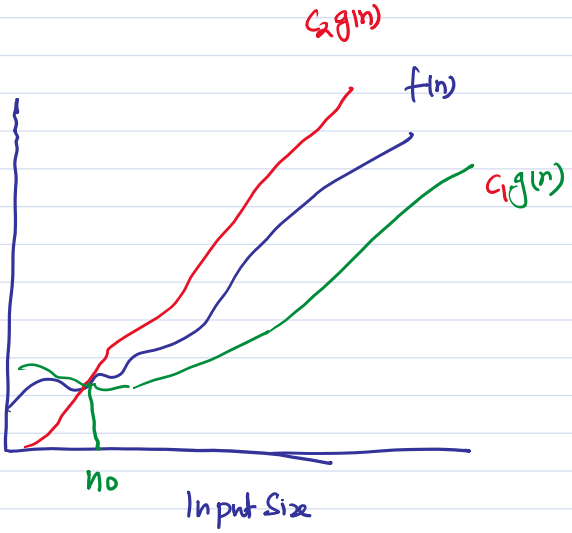
$f(n) = \Omega(n^2)$.

f is atleast as large as g

Asymptotic tight Bound (Θ Notation)

$f(n) = \Theta(g(n))$ if const c_1 and c_2 and n_0 s.t

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$$



$f(n) = 10n^2 + 5n + 100$

$10n^2 \leq f(n) \leq 115\, n^2 \quad \forall n \geq n_0 = 1$

$f(n) = \Theta(n^2)$

Little oh notation

$f(n) = o(g(n))$

$\forall c \quad \exists n_0$ s.t $\quad f(n) < c\, g(n) \quad \forall n \geq n_0$

$\equiv$

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

Ex $\quad f(n) = \log n \qquad f(n) = o(n)$

Ex $\quad f(n) = \sqrt{n} \qquad g(n) = n \qquad \sqrt{n} = o(n)$

$f(n) = n^{1-\varepsilon} \qquad \varepsilon > 0 \qquad f(n) = o(n)$

"f is strictly smaller than g"

Small Omega notation

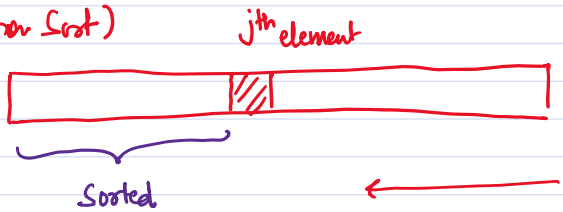
$f(n) = \omega(g(n))$

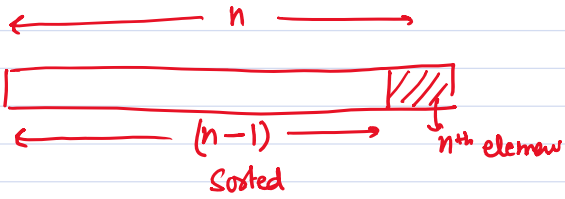
$\forall c \quad \exists n_0$ s.t $\quad c \cdot g(n) < f(n) \quad \forall n \geq n_0$

$$\lim_{n \to \infty} \frac{g(n)}{f(n)} = 0$$

"f is strictly larger than g".

Recurrance Relation (Insertion Sort)





Worst Case

$$T(n) = T(n-1) + C(n-1)$$

$$= T(n-2) + C(n-2) + C(n-1)$$

$$= T(n-3) + C(n-3) + C(n-2) + C(n-1)$$

$$= T(1) + C[2 + 3 + \cdots (n-3) + (n-2) + (n-1)]$$

$$= C' + C[2 + 3 + \cdots (n-3) + (n-2) + (n-1)]$$

$T(n) = \Theta(n^2)$

↑ Worst case running time of insertion sort

Best Case

$$T(n) = T(n-1) + C$$

↑ Best case running time of insertion sort

$$T(n) = T(n-2) + C + C$$

$$= T(n-3) + C + C + C$$

$$\vdots$$

$$= T(1) + (n-1)C$$

$$T(n) = C'' + (n-1)C \quad (\text{let } T(1) = C'')$$

$$T(n) = \Theta(n)$$

Avg case $T(n) = T(n-1) + n/2$

For recurrence relation $T(n) = T(n')$ $n' < n$